



**CS 315**

**Programming Languages**

**Project 2 Report**

**Team 2**

Team Members:

Atakan Akar 22203140 Section-1

Hüseyin Deniz Kurtulan 22203720 Section-1

Kerem Varnalı 22202788 Section-1

Language Name: **DaK#**

## Table of Contents

|                                                            |           |
|------------------------------------------------------------|-----------|
| <i>The Language Constructs &amp; BNF Descriptions.....</i> | <b>3</b>  |
| <i>Non-Trivial Tokens Meanings:.....</i>                   | <b>7</b>  |
| <i>Complete BNF of the Program:.....</i>                   | <b>9</b>  |
| <i>Language Criteria: .....</i>                            | <b>13</b> |
| <i>Operation Precedence &amp; Associativity.....</i>       | <b>14</b> |
| <i>TYPE CHECKING RULES: .....</i>                          | <b>16</b> |
| <i>Type Conversion Rules: .....</i>                        | <b>17</b> |

## The Language Constructs & BNF Descriptions

**<program>:** This non-terminal indicates the beginning non-terminal of our language “DaK”. Valid DaK# derivation begins from this non-terminal. Every program that is written in DaK# start with <dak (which is <start\_symbol>) and ends with #> (which is the <end\_symbol>).

**<stmt\_list>:** This non-terminal can be divided into two possibilities. One of which can be just one statement that ends with semicolon or also one statement with semicolon followed by more statements.

**<stmt>:** Represents a statement in a statement list. It can branch to a if statement and a casual statement.

**<if\_stmt>:** A conditional block starting with if, evaluating an <expr>, followed by a <stmt\_list> enclosed in begin and end. Optionally chains to <elif\_case> (additional conditions) or <else\_case> (default block) for multi-branch logic. However, it may contain only an if statement.

**<elif\_case>:** Extends an if statement with one or more elif branches, each checking a new <logic\_expr> and executing its own <stmt\_list>. Can further chain to other <elif\_case>, <else\_case> blocks, or it may not chain anywhere and stmt\_list may continue.

**<else\_case>:** A default block executed when preceding if or elif conditions fail, defined with else begin ... end and containing a <stmt\_list>.

**<casual\_stmt>:** General statements including variable declarations, assignments, loops, jumps (like return), io commands, comments, or function definitions. Terminated with ; unless nested within a begin and end.

**<declaration\_stmt>:** Declares variables of a <data\_type>, either as a single identifier, a comma-separated <identifier\_list>, or an array (e.g., int x[5]). Does not initialize values.

**<identifier\_list>:** A comma-separated list of valid identifiers (variable/function names), allowing bulk declaration or parameter definitions in functions.

**<identifier>:** A user-defined name starting with a letter, followed by letters, digits, or underscores. Used for variables, functions, or array accesses.

**<identifier\_tail>:** Completes an identifier after its initial letter by recursively allowing additional letters, digits, or underscores.

**<digit>:** Represents single digit which are 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9.

**<letter>:** Handles all of the letters in the alphabet.

**<init\_stmt>:** Declares and initializes variables in one step, supporting direct assignment (e.g., int x = 5), input assignments (e.g., input(...)), or array initialization with begin ... end.

**<array\_arg\_list>:** A list of values (constants or identifiers) used to initialize an array, enclosed in begin and end.

**<assign\_stmt>:** Defines assignments to variables or array elements. It supports: assigning an <expr> to an <identifier> using <all\_assign\_ops>; chaining assignments (e.g.,  $x = y = 5$ ); assigning user input (`input(<const_string>)`); and assigning to array elements (<array\_access>). Enables flexible value updates, including compound operations.

**<all\_assign\_ops>:** Represents assignment operators: `=` for basic assignment, `+=` for addition assignment, `-=` for subtraction assignment, `*=` for multiplication assignment, and `/=` for division assignment. Combines arithmetic and assignment in one step, simplifying variable updates.

**<array\_access>:** Accesses an array element via an index (e.g., `arr[x+1]`), where the index is a <math\_expr>.

**<expr>:** A generic expression, starts with a logical (<logic\_expr>) and branches to arithmetic (<math\_expr>) operations. Forms the basis of conditions, calculations, or assignments.

**<logic\_expr>:** A boolean expression built from logical operators (`||`, `&&`, `!`), relational operators (`<`, `>`, `==`, etc.), and operands like identifiers, constants, or function calls. Evaluates to true or false. Also it branches to arithmetic (<math\_expr>) operations. However at the end logic expressions and arithmetic expressions may mix up and situations such as `7 + true` may occur. In the BNF and Lex they may be differentiate but in this scenario some ambiguity can occur. Therefore this type of mix ups will be solved with parser (Yacc).

**<or\_expr>:** Executes OR operations by `||` symbol. Also it directs to AND operations.

**<and\_expr>:** Performs AND operations with `&&` symbol. This operation branches to equality operations.

**<equality\_expr>:** Handles equality (`==`) and inequality (`!=`) operations. From this expression, comparison expressions are branched.

**<relation\_expr>:** Covers all of the comparison operators such as `>`, `<`, `>=`, `<=`. This operation directs to not expressions.

**<not\_expr>:** Executes NOT operations by `!` symbol. Also this is the last logical operation. After this operation arithmetic expression are branched.

**<math\_expr>:** A numeric expression involving arithmetic operations (`+`, `-`, `*`, `/`, `%`, `^`), unary operators, parentheses, identifiers, constants, or function calls. Resolves to integer or float values.

**<addition\_expr>:** First mathematical expression and it has lowest precedence among all of the arithmetic operations. This expression captures addition (`+`) and subtraction (`-`) operations

**<mult\_expr>:** Handles multiplication (\*), division (/) and (%) operations. Also it branches to power expressions.

**<power\_expr>:** Performs to take power operations by ^ symbol. From this operation unary expression is derived.

**<unary\_expr>:** This expression decides the sign of a variable or a constant if its sign is defined. In this way positive sign and addition sign is differentiated.

**<basic\_expr>:** This expression branches into five items. First one is paranthesized expressions then identifier, constants, function call and finally array acces.

**<loop\_stmt>:** Defines loops: while loops with a condition, or for loops with optional initialization, condition, and increment/decrement clauses. Executes a <stmt\_list> enclosed in begin ... end.

**<jump\_stmt>:** Alters control flow with return (optionally with a value), break, or continue. Exits functions, terminates loops, or skips iterations.

**<print\_stmt>:** Outputs values to the console via print(...), accepting a <list\_to\_printing> of expressions.

**<list\_to\_printing>:** This expression allows a comma-separated sequence of <to\_print> items, enabling the print statement to output multiple expressions in one call.

**<to\_print>:** Represents a single expression to be printed.

**<function\_declaration>:** Defines a function with func, a name, optional <param\_list>, and a <stmt\_list> body in begin ... end. Parameters specify <data\_type> and identifier.

**<function\_call>:** Invokes a function by name with arguments in parentheses (e.g., foo(5, x)). Arguments are <expr> values passed to the function.

**<data\_type>:** Specifies variable or parameter types: int, float, bool, char, or string. Determines data storage and operations.

**<constant>:** Literal values like integers, floats, booleans, characters, or strings. Includes syntax for quotes (e.g., "hello", 'a', true).

**<comment>:** Inline (//...) or block (/\* ... \*/) comments ignored during execution.

**<start\_symbol> and <end\_symbol>:** Delimiters <dak and #> marking the start and end of a program.

**Operators (<op\_or>, <op\_and>, <op\_equality>, <op\_inequality>, <op\_less\_than>, <op\_greater\_than>, <op\_less\_than\_or\_equal>, <op\_greater\_than\_or\_equal>, <op\_addition>, <op\_subtraction>, <op\_multiplication>, <op\_division>, <op\_assignment>, <op\_modulo>):** Symbols like +, -, &&, ^, etc., defining arithmetic,

logical, or assignment operations. Follow precedence rules in expressions eliminating ambiguity..

**<param\_list>:** Comma-separated function parameters, each with a <data\_type> and <identifier>.

**<param>:** Represents a single parameter of a param list.

**<arg\_list>:** Comma-separated arguments passed during a function call, evaluated as <expr> values.

**<const\_string>:** A string literal enclosed in quotes, containing characters, digits, or symbols. Constructed with <string\_contents>

**<string\_contents>:** This non-terminal is used within the <const\_string>. It recursively defines the sequence of allowed characters inside a string literal. It can represent an empty sequence or one or more <character> elements, ensuring that the contents of a string are properly structured.

**<character>:** This defines a single character that can appear in string literals or character constants. It may be a letter, digit, or any symbol as defined by <symbol>. This non-terminal ensures that every character is recognized as a valid building block for textual constructs.

**<symbol>:** Specifies the set of special characters (such as punctuation marks and other non-alphanumeric symbols) that are allowed in the language. This set is used when constructing <character> tokens and ensures that symbols appearing in strings or identifiers meet the language's requirements.

**<const\_char>:** A single character enclosed in single quotes.

**<const\_float>:** A floating-point number with a decimal point (e.g., 3.14 or .5).

**<const\_int>:** An integer literal composed of digits.

**<const\_bool>:** The boolean literals true or false.

## Non-Trivial Tokens Meanings:

### Data Type Keywords

**int:** Describes type Integer for identifiers and functions.

**string:** Describes type String for identifiers and functions.

**bool:** Describes type String for identifiers and functions.

**char:** Describes type Char for identifiers and functions.

**float:** Describes type Float for identifiers and functions.

### Boolean Literals

**true:** Represents the boolean value "true".

**false:** Represents the boolean value "false".

### Reserved Operation Symbols

**/ :** Division operator.

**%:** Modulo operator.

**||:** Or operator.

**&&:** And operator.

**==:** Equality operator.

**!=:** Inequality operator.

**<:** Less than operator.

**>:** Greater than operator.

**<=:** Less than or equal operator.

**>=:** Greater than or equal operator.

**!:** Not operator.

**^:** Power operator.

**+=:** Add and assign operator.

**-=:** Subtract and assign operator.

**\*=:** Multiply and assign operator.

**/=:** Divide and assign operator.

**//:** Line comment.

**/\*:** Block comment start.

**\*/:** Block comment end.

### Control Flow Keywords

**if:** Begins a conditional statement. Executes a block of code if the given condition is true.

**elif:** (Short for "else if") Adds an additional condition to an if statement.

**else:** Defines an alternative block to execute if all preceding if and elif conditions are false.

### Looping Keywords

**while:** Defines a loop that executes while the given condition remains true.

**for:** Defines a loop with an initialization, a condition, and an update expression.

### Loop Control Keywords

**return:** Exits from a function and optionally returns a value.

**continue:** Skips the rest of the loop body and moves to the next iteration.

**break:** Exits the current loop immediately.

### Block Structure Keywords

**begin:** Starts a block of code for control structures and functions.

**end:** Ends a block of code for control structures and functions.

### Function Keywords

**func:** Declares a function.

**print:** Outputs values to the console.

**input:** Reads input from the user.

## Complete BNF of the Program:

```

<program> ::= <start_symbol> <stmt_list> <end_symbol>
<stmt_list> ::= <stmt> | <stmt> <stmt_list>
<stmt> ::= <if_stmt> | <casual_stmt>
<if_stmt> ::= if (expr) begin <stmt_list> end
           | if (expr) begin <stmt_list> end <elif_case>
           | if (expr) begin <stmt_list> end <else_case>
<elif_case> ::= elif (expr) begin <stmt_list> end
           | elif (expr) begin <stmt_list> end <elif_case>
           | elif (expr) begin <stmt_list> end <else_case>
<else_case> ::= else begin <stmt_list> end
<casual_stmt> ::= <declaration_stmt> ;
           | <init_stmt>
           | <assign_stmt> ;
           | <loop_stmt> ;
           | <jump_stmt> ;
           | <comment>
           | <print_stmt> ;
           | <function_declaration>
           | <function_call> ;
<declaration_stmt> ::= <data_type> <identifier>
           | <data_type> <identifier> [<const_int>]
<init_stmt> ::= <data_type> <identifier_list> <op_assignment> <expr>
           | <data_type> <identifier_list> <op_assignment> input(<const_string>)
           | <data_type> <identifier> [<const_int>] <op_assignment> begin <array_arg_list>
end
<array_arg_list> ::= <expr> <array_arg_list>
           | <expr>
<assign_stmt> ::= <identifier> <all_assign_ops> <expr>
           | <identifier> <all_assign_ops> <assign_stmt>
           | <identifier> <op_assignment> input(<const_string>)
           | <array_access> <all_assign_ops> <expr>
           | <array_access> <op_assignment> input(<const_string>)

```

```

<all_assign_ops> ::= <op_assignment>
    | <op_assign_plus>
    | <op_assign_minus>
    | <op_assign_multiply>
    | <op_assign_divide>
<array_access> ::= <identifier> [<expr>]
<expr> ::= <logic_expr>
<logic_expr> ::= <or_expr>
<or_expr> ::= <or_expr> <op_or> <and_expr>
    | <and_expr>
<and_expr> ::= <and_expr> <op_and> <equality_expr>
    | <equality_expr>
<equality_expr> ::= <equality_expr> <op_equality> <relation_expr>
    | <equality_expr> <op_inequality> <relation_expr>
    | <relation_expr>
<relation_expr> ::= <relation_expr> <op_less_than> <not_expr>
    | <relation_expr> <op_greater_than> <not_expr>
    | <relation_expr> <op_less_than_or_equal> <not_expr>
    | <relation_expr> <op_greater_than_or_equal> <not_expr>
    | <not_expr>
<not_expr> ::= <op_not> <not_expr>
    | <math_expr>
<math_expr> ::= <addition_expr>
<addition_expr> ::= <addition_expr> <op_addition> <mult_expr>
    | <addition_expr> <op_subtraction> <mult_expr>
    | <mult_expr>
<mult_expr> ::= <mult_expr> <op_multiplication> <power_expr>
    | <mult_expr> <op_division> <power_expr>
    | <mult_expr> <op_modulo> <power_expr>
    | <power_expr>
<power_expr> ::= <power_expr> <op_power> <unary_expr>
    | <unary_expr>
<unary_expr> ::= <op_addition> <unary_expr>
    | <op_subtraction> <unary_expr>

```

```

    | <basic_expr>
<basic_expr> ::= (<expr>)
    | <identifier>
    | <constant>
    | <function_call>
    | <array_access>
<loop_stmt> ::= while (<expr>) begin <stmt_list> end
    | for (<init_stmt>; <expr>; <assign_stmt>) begin <stmt_list> end
    | for (<assign_stmt>; <expr>; <assign_stmt>) begin <stmt_list> end
    | for (; <expr>; <assign_stmt>) begin <stmt_list> end
    | for (; <expr>; ) begin <stmt_list> end
<jump_stmt> ::= return <expr> | return | continue | break
<print_stmt> ::= print (<list_to_printing>)
<list_to_printing> ::= <to_print>
    | <to_print> , <list_to_printing>
<to_print> ::= <expr>
<function_declaration> ::= func <identifier>() begin <stmt_list> end
    | func <identifier> (param_list) begin <stmt_list> end
<param_list> ::= <param> |
    <param> , <param_list>
<param> ::= <data_type> <identifier>
<function_call> ::= <identifier> () |
    <identifier> (arg_list)
arg_list ::= <expr> |
    <expr> , <arg_list>
<data_type> ::= int | float | bool | char | string
<character> ::= <letter> | <digit> | <symbol>
<letter>
::= a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t | u |
v |
w | x | y | z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P |
Q | R | S | T | U | V | W | X | Y | Z
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

```

```

<symbol>
::= # | $ | % | & | [ | ] | ^ | _ | ` | { | | | } | ~ | * | ( | ) | + | , | ! | - | . | /
| \ | : | ; | < | = | > | ? | @
<start_symbol> ::= <DaK
<end_symbol> ::= #>
<constant> ::= <const_int>
                | <const_float>
                | <const_bool>
                | <const_char>
                | <const_string>
<const_string> ::= "\"" <string_contents> "\""
<string_contents> ::= "" | <character> <string_contents>
<const_int> ::= <digit> | <const_int> <digit>
<const_float> ::= <const_int> . <const_int>
                | . <const_int>
<const_char> ::= '<character>'
<const_bool> ::= true | false
<comment> ::= <comment_line> | <comment_block>
<op_assignment> ::= =
<op_addition> ::= +
<op_subtraction> ::= -
<op_multiplication> ::= *
<op_division> ::= /
<op_modulo> ::= %
<op_or> ::= ||
<op_and> ::= &&
<op_equality> ::= ==
<op_inequality> ::= !=
<op_less_than> ::= <
<op_greater_than> ::= >
<op_less_than_or_equal> ::= <=
<op_greater_than_or_equal> ::= >=
<op_not> ::= !
<op_power> ::= ^

```

```

<op_assign_plus> ::= +=
<op_assign_minus> ::= -=
<op_assign_multiply> ::= *=
<op_assign_divide> ::= /=

```

## Language Criteria:

### A.Writability:

**Syntax Complexity:** We designed our syntax to be consistent and simple ensuring easier code writing.

**For loops:** Allowing multiple forms of the for loop enhances writeability by providing flexibility, enabling more concise code, and allowing programmers to choose the most suitable structure for their needs. It also improves efficiency by reducing unnecessary code and aligning with familiar C-style syntax, making it easier to write loops.

**Comprehensive Conditional Statements:** The if-elif-else structure enables clear and structured decision-making. This improves code readability by reducing nested conditions and provides a logical flow, making complex decision-making easier to express and maintain.

**C-like Operators:** (=, +, -, \*, /, ==, !=, etc.) make the language familiar to programmers experienced in C-style languages.

**Compound Assignment Operators:** The inclusion of **compound assignment operators** (**+=**, **-=**, **\*=**, **/=**) further enhances writability by allowing shorthand updates to variables, eliminating redundancy. These features speed up coding, reduce errors, and make modifying expressions more efficient, ultimately improving the developer's workflow.

### B.Readability:

**Begin-end (Reserved keywords):** Instead of relying solely on indentation or parentheses, using **begin** and **end** as reserved keywords improves readability by explicitly marking the start and end of code blocks, making it easy to follow program structure. This consistency across functions, loops, and conditionals ensures a predictable syntax, reducing ambiguity. Additionally, in deeply nested structures, **begin** and **end** provide clear visual separation, preventing confusion and enhancing code clarity.

**Variable Declarations:** Variable declarations require explicit type annotations (int, float, bool, etc.), which allows programmers to immediately recognize data types and supports code readability.

**Logical Operator Symbols:** Using conventional symbols (similar to the C-like language) for logical operations (`||`, `&&`, `==`, `!=`, `<=`, `>=`) improves readability by maintaining consistency with other programming languages, making conditional expressions easier to understand. Additionally, they follow standard precedence rules, making logical expressions predictable and reducing errors during debugging.

### **Type Conversions:**

Standard type conversions such as “1 is evaluated as true” and “0 is evaluated as false” when boolean value is expected but integer is given enhance which is a common practice in many popular programming languages that developers are used to it. However, some of the type conversions such as empty string might be evaluated as false and other strings are evaluated as true when boolean value is expected but string type is given might reduce the readability criteria.

### **C. Reliability:**

**Clear, Explicit Grammar:** DaK’s syntax is thoroughly described via a formal BNF, making it easier to parse and less prone to ambiguous interpretations. This helps developers and compiler writers reliably understand the structure of DaK programs.

**Straightforward Control Structures:** With keywords like `if`, `else`, `for`, `while`, `func`, etc., DaK mirrors many established imperative languages. These features, clearly organized in the grammar, lower the learning curve and reduce the chance of logical confusion.

**Consistent Operator Precedence:** The grammar explicitly defines the precedence and associativity of arithmetic, relational, and logical operators. This consistency helps ensure that expressions behave predictably and as developers would expect, enhancing reliability in expression evaluation.

## **Operation Precedence & Associativity**

### **1) Primary Expressions**

- **Operators/Forms:** Parenthesized expressions ( `<expr>` ), identifiers, constants, function calls, array accesses.

### **2) Unary Operators**

- **Operators:** Unary plus (+), unary minus (-), logical NOT (!)
- **Associativity:** Right-to-left

- **Note:** In the grammar, these appear in `<unary_expr>` (for + and -) and `<not_expr>` (for !).

### 3) Exponentiation

- **Operator:** Power (^)
- **Associativity:** Right-to-left
- **Grammar Location:** `<power_expr>` is defined recursively with right associativity.

### 4) Multiplicative Operators

- **Operators:** Multiplication (\*), Division (/), Modulo (%)
- **Associativity:** Left-to-right
- **Grammar Location:** Found in `<mult_expr>`.

### 5) Additive Operators

- **Operators:** Binary addition (+), Binary subtraction (-)
- **Associativity:** Left-to-right
- **Grammar Location:** Defined in `<addition_expr>`.

### 7) Relational Operators

- **Operators:** Less than (<), Greater than (>), Less than or equal (<=), Greater than or equal (>=)
- **Associativity:** Left-to-right
- **Grammar Location:** In `<relation_expr>`.

### 8) Equality Operators

- **Operators:** Equality (==), Inequality (!=)
- **Associativity:** Left-to-right
- **Grammar Location:** In `<equality_expr>`.

### 9) Logical AND

- **Operator:** Logical AND (&&)
- **Associativity:** Left-to-right
- **Grammar Location:** In `<and_expr>`.

## 10) Logical OR

- **Operator:** Logical OR (||)
- **Associativity:** Left-to-right
- **Grammar Location:** In <or\_expr>.

## 11) Assignment Operators

- **Operators:** Simple assignment (=) and compound assignments (+=, -=, \*=, /=)
- **Associativity:** Right-to-left
- **Note:** Although assignment is not part of the <expr> production (it's handled in <assign\_stmt> and <init\_stmt>), it is typically given the lowest precedence so that the right-hand side is evaluated before assignment.

## Type Checking Rules:

1. **Arithmetic operations (+, -, \*, /, ^, %, +=, -=, \*=, /=):**
  - If an arithmetic operation includes any float, all of the given types are converted to float. Returns float.
  - If an arithmetic operation does not include any float, all of the given types are converted to int. Returns int.
2. **Comparison operations (<, >, <=, >=, ==, !=):**
  - All of the given types are converted to float. Returns bool.
3. **Array indices (array[i]):**
  - Given index number is converted to int. Returns element of the array.
  - If the converted int is negative or the given int is negative, it starts counting from the end of the array.
4. **Logic operations (&&, ||, !):**
  - All of the given types are converted to bool. Returns bool.
5. **Print operations (print("Hello World")):**
  - All of the given types are converted to str. Print the given items.
6. **Array initialization:**
  - All of the array elements are converted to the declared type of the array.

## Type Conversion Rules:

### Conversion to Bool:

When a number (float or int) is given, if the number is 0, that value is cast as **false**, otherwise it is cast as **true**.

- `0 || 1.5` is considered as **false || true = true**
- Strings are considered false if empty and true if they contain any characters.

Examples:

- `7 && 5` means evaluates to true && true = true
- `"" || "test"` evaluates to false || true = true

### Conversion to Int:

- Booleans are converted to 0 if false and 1 if true.
- When a string is given, if it is an empty string, it is considered as 0 otherwise if it has any characters in it, it is considered as 1.
- When a float is given, only the integer places are taken.

Examples:

- `false + true` evaluates to `0 + 1 = 1`
- `"test" + 5 = 6`
- `3 + "str"` results 4
- `arr[0.5]` is evaluated as `arr[0]`

### Conversion to Str:

- If a string is required but a different type is given, the value is simply converted into a string representation.

Examples:

- `false` results in `"false"`
- `5.4` results in `"5.4"`
- `4` results in `"4"`

### Conversion to Float:

- Booleans are converted to 0.0 if false and 1.0 if true.
- When a string is given, if it is an empty string, it is considered as 0.0 otherwise if it has any characters in it, it is considered as 1.0.
- When an int is given, only decimal places are added.

Examples:

- $1.0 + 5 = 6.0$
- $3.0 + \text{"str"}$  results in 4.0